

Solutions to Chapter 3 Exercises (Sutton & Barto, Reinforcement Learning)

Exercise 3.1: Three Example Tasks in the MDP Framework

Example 1: Autonomous Delivery Drone

- **States:** Drone’s location, battery level, weather conditions.
- **Actions:** Move North, South, East, West; Deliver Package; Recharge.
- **Rewards:** +10 for successful delivery, -1 for each movement, -20 for crashing due to low battery.

Example 2: Personalized Learning Assistant

- **States:** Student’s current skill level and topic understanding.
- **Actions:** Recommend video, quiz, explanation, or take a break.
- **Rewards:** +5 for improvement in quiz performance, -1 for disengagement, +1 for staying engaged.

Example 3: Urban Traffic Light Control (Stretching MDP)

- **States:** Vehicle counts at intersections, time of day.
- **Actions:** Change light phase, extend green, switch to red.
- **Rewards:** -1 for congestion, +5 for smooth flow, -10 for accidents.

Exercise 3.2: Limits of the MDP Framework

The MDP framework is quite flexible but assumes full observability and a stationary environment. It struggles with:

- **Partially Observable Tasks:** e.g., Poker, where agents don’t see full state.
- **Non-stationary Goals:** e.g., Stock markets reacting to rare events.

Thus, while powerful, MDPs may not adequately model all real-world learning problems.

Exercise 3.3: Choosing the Agent-Environment Boundary in Driving

The correct level depends on:

- **Abstraction and Feasibility:** Defining actions as high-level driving goals (e.g., “drive to office”) may simplify learning.
- **Data Availability and Control:** Where we can observe and influence reliably (e.g., steering/braking).

No fundamental rule exists—the choice is task-dependent and balances abstraction, clarity, and control over outcomes.

Exercise 3.4: Table for $p(s', r \mid s, a)$

s	a	s'	r	$p(s', r \mid s, a)$
High	Search	High	1	0.9
High	Search	Low	1	0.1
High	Wait	High	0	1.0
Low	Search	Low	1	0.2
Low	Search	Low	0	0.8
Low	Wait	Low	0	1.0
Low	Recharge	High	0	1.0

0.1

Exercise 3.7 **Answer:** The robot receives a reward of +1 only upon escaping and 0 at all other times. As a result, no matter how long the robot takes to escape, the total reward it receives per episode is always 1. This means that the agent has no incentive to improve or minimize the time it takes to escape the maze because all episodes yield the same return. The issue here is that the agent is not receiving any feedback that distinguishes between fast and slow escapes. As a result, the agent does not learn a preference for faster solutions. To better communicate the objective of escaping quickly, one can: Penalize time steps by assigning a small negative reward (e.g., \$-0.01\$) at each step to encourage quicker escapes. Use discounting to value earlier rewards more highly, which naturally encourages faster goal achievement. **0.2**

Exercise 3.16 Now consider adding a constant c to all the rewards in an episodic task, such as maze running. Would this have any effect, or would it leave the task unchanged as in the continuing task above? Why or why not? Give an example. **Answer:** In episodic tasks, adding a constant c to all rewards **does change the task**. This is because the total return becomes:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T$$

If we add a constant c to each reward, the return becomes:

$$G'_t = G_t + c(T - t)$$

This added term $c(T - t)$ depends on how long the episode lasts. So, longer episodes would give higher returns just because of the added constant. This can change the agent's behavior. **Example:** In a maze task, if the agent gets +1 for escaping and 0 otherwise, adding $c = 0.1$ to all rewards could make the agent delay escaping to collect more of the small constant rewards. So the task is no longer about escaping quickly. **Exercise 3.20**

Answer: Changing the policy π changes the action selection probabilities $\pi(a|s)$, which directly affects the value function $v_\pi(s)$. Even a small change in the policy can lead to a significant difference in the value function, especially in long-horizon tasks or tasks where particular actions lead to very different outcomes. This is because the value function aggregates future rewards based on the behavior dictated by the policy. Hence, the value function is sensitive to the policy, and $v_{\pi_1}(s) \neq v_{\pi_2}(s)$ in general when $\pi_1 \neq \pi_2$. **Exercise 3.21**

Answer: Yes, it is possible for two policies to have the same state-value function $v_\pi(s)$ but different action-value functions $q_\pi(s, a)$. This can happen when the policies differ in how they distribute probabilities over actions in certain states, yet the expected value $v_\pi(s)$, which is the average over actions weighted by $\pi(a|s)$, remains the same. Mathematically:

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s, a)$$

Different $\pi(a|s)$ and $q_\pi(s, a)$ combinations can yield the same $v_\pi(s)$. **Exercise 3.22**

Answer: No, a policy being greedy with respect to its own action-value function q_π does not guarantee optimality. That is, if:

$$\pi(s) = \arg \max_a q_\pi(s, a)$$

this does not imply that π is the optimal policy π^* . The reason is that $q_\pi(s, a)$ reflects the expected return if the policy π is followed thereafter, not necessarily the best possible return. A truly optimal policy π^* satisfies: